Unit Testing in C# with NUnit

Zed editor | .NET 10 | Command-line workflow

Learning outcomes: create two linked C# projects; write NUnit tests; use SetUp, Test and assertions; run tests and interpret failures.

1. Create a folder and two projects

Open a terminal in a folder where you have write access. Run the commands one at a time:

```
mkdir Codebase_Unit_test
cd Codebase_Unit_test
dotnet new sln -n UnitTestingDemo
dotnet new console -n Codebase
dotnet new nunit -n UnitTests
dotnet sln add Codebase/Codebase.csproj
dotnet sln add UnitTests/UnitTests.csproj
```

Open the Codebase_Unit_test folder in Zed. The expected structure is:

```
Codebase_Unit_test/
  Codebase/
    Program.cs
    Codebase.csproj
  UnitTests/
    UnitTests.csproj
    UnitTest1.cs  (or another generated test filename)
```

2. Link UnitTests to Codebase

A test project must reference the project whose methods it tests. From the root folder(Codebase_Unit_test) , run:

```
dotnet add UnitTests/UnitTests.csproj reference Codebase/Codebase.csproj
```

This adds a ProjectReference to UnitTests.csproj. The arrow in the slide points from the test project to the codebase project, not the other way around.

3. Understand UnitTests.csproj

Open UnitTests/UnitTests.csproj in Zed. The key concepts are:

- TargetFramework: the .NET framework for the test project (for example net10.0).
- ProjectReference: points to ../Codebase/Codebase.csproj.

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <LangVersion>latest</LangVersion>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <IsPackable>false</IsPackable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="coverlet.collector" Version="6.0.4" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.14.0" />
    <PackageReference Include="NUnit" Version="4.3.2" />
    <PackageReference Include="NUnit.Analyzers" Version="4.7.0" />
    <PackageReference Include="NUnit3TestAdapter" Version="5.0.0" />
  </ItemGroup>

  <ItemGroup>
    <Using Include="NUnit.Framework" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\Codebase\Codebase.csproj" />
  </ItemGroup>

</Project>

```

4. Create the Codebase/Util.cs file

In Zed, create a new file named Util.cs inside Codebase and paste:

```
namespace Codebase;

public class Util
{
    public static int Add(int a, int b)
    {
        return a + b;
    }
}
```

The method is static, so tests can call Util.Add(1, 2) without creating a Util object.

5. Update Codebase/Program.cs

Open a Program.Cs file in zed and paste :

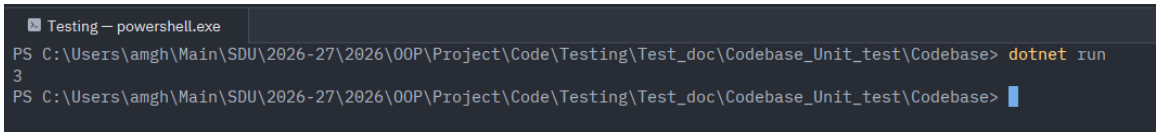
```
using Codebase;

public class Program
{
    public static void Main()
    {
        Console.WriteLine(Util.Add(1, 2));
    }
}
```

Save the file and then in zed terminal Run:

dotnet run

Expected console output: 3.



6. Create NUnit tests in UnitTests

Replace the generated test file with the following code :

```
using Codebase;
using NUnit.Framework;

namespace UnitTests;
public class UtilTests
{
    [SetUp]
    public void Setup()
    {
        // Runs before each test. No setup needed yet.
    }

    [Test]
    public void Add_OneAndTwo_ReturnsThree()
    {
        // Arrange
        int a = 1;
        int b = 2;

        // Act
        int result = Util.Add(a, b);

        // Assert
        Assert.That(result, Is.EqualTo(3));
    }
}
```

7. Run the test in the Zed terminal

From the root project folder:

```
dotnet test UnitTests/UnitTests.csproj
```

If tests are discovered and run, the summary should show 1 passed, 0 failed. If you are inside UnitTests already, use dotnet test. You can also run dotnet test from the solution root.

